

DSA Study Notes

Generated from the uploaded Chapter 1-9 lecture slides

Includes ADT, STL/vector, linked lists, stacks, queues, trees, graphs, sorting, and searching.

DSA Study Notes From Uploaded Slides

- Source basis:
 - Chapter 1 Abstract Data Type and Introduction to STL (Part 1)
 - Chapter 2 Complexity Analysis
 - Chapter 3 Linked Lists (Part 1)
 - Chapter 3 Linked Lists (Part 2)
 - Chapter 4 Stacks
 - Chapter 5 Queues
 - Chapter 6 Tree (Part 1)
 - Chapter 6 Tree (Part 2)
 - Chapter 7 Graph
 - Chapter 8 Sorting
 - Chapter 9 Searching
- Study-rule acknowledgement:
 - These notes follow the chapter order in the uploaded decks.
 - The notes are based on the slide content.
 - Extra C++ code is included only to explain slide concepts fully, because the slides often show partial code or pseudocode.
 - Slide-code corrections are marked when needed.
 - Paragraphs are kept short and bullets are used heavily for revision.

Chapter 1: Abstract Data Type and Introduction to STL

Learning Targets

- Understand:
 - Data structures
 - Abstract Data Types (ADTs)
 - Standard Template Library (STL)
 - Containers, iterators, and algorithms
 - Vector/list mentions in STL context

1.1 Data Structure

- A program is made of:
 - Data
 - Algorithms
- A data structure is:
 - A logical or mathematical model for organizing data
 - A technique for storing and organizing data efficiently
 - A way to describe relationships among elements, not only memory storage
- Choosing a data structure depends on:
 - Whether it represents real-world relationships well
 - Whether it is simple enough to process effectively
- Why learn data structures:
 - They make algorithms simpler
 - They make code easier to maintain
 - They often make programs faster

1.2 Core Data Operations

- Main operations from the slides:
 - Insert
 - Delete
 - Traverse
 - Search
- Additional special operations:
 - Sort
 - Merge

1.3 ADT: Abstract Data Type

- Definition:
 - An ADT is a mathematical model for data types.
 - It defines behavior from the user's point of view.
 - It focuses on:
 - Possible values
 - Possible operations

- Behavior of those operations
- Important idea:
 - An ADT tells you what can be done.
 - It does not tell you how it is implemented.
- Examples from the slides:
 - Array
 - List
 - Queue
 - Stack
 - Table
 - Tree
 - Vector

ADT Memory Aid: "VOB"

- V = Values allowed
- O = Operations allowed
- B = Behavior of those operations
- Exam phrase:
 - "An ADT defines VOB, not code."

1.4 ADT Example: Stack

- Stack ADT behavior:
 - Last In First Out (LIFO)
 - Items are added and removed from one end only
- Stack operations from the slides:
 - PUSH: add an item
 - POP: remove the top item
 - TOP: return the top item without removing it
 - EMPTY: check whether the stack is empty
 - CREATE: create a new empty stack
- ADT point:
 - These operations describe what the stack does.
 - Implementation details are saved for the actual code.

1.5 STL: Standard Template Library

- STL provides generic entities:
 - Containers
 - Iterators
 - Algorithms
 - Function objects
- STL also provides ready-made common C++ classes:
 - Containers
 - Associative arrays

- STL can be used with:
 - Built-in types
 - User-defined types that support required operations
- STL uses templates:
 - This gives static binding polymorphism.
 - The slides state this is often more efficient than run-time/dynamic binding.

1.6 STL Containers

- A container is:
 - A data structure designed to hold objects of the same type
 - A way stored data is organized in memory
 - Implemented as a template class
- Container methods:
 - Some methods are common to all containers.
 - Some are specific to one container.
- If pointers are involved:
 - Data stored in containers must support the needed basic methods and operations.

Code Mastery: Simple Node Container-Style Class

- Understand it:
 - The slides show a class that creates a node dynamically.
 - The node stores data and a pointer to the next node.
- Read it:
 - `Node *next` means the node stores an address of another node.
 - `new Node()` allocates memory dynamically.
 - `n->data` accesses a member through a pointer.
 - `NULL` marks no next node.
- Write it:

```
#include <iostream>
using namespace std;

class MyClass {
    struct Node {
        int data;
        Node *next;
    };

    Node *head;

public:
    MyClass() {
        head = NULL;
    }

    Node* createNode() {
        Node *n = new Node();
        cout << "Enter a number: ";
        cin >> n->data;
        n->next = NULL;
        return n;
    }
};

int main() {
    MyClass record;
    record.createNode();
}
```

```
    return 0;
}
```

1.7 STL Iterators

- An iterator is:
 - An object that accesses container elements
 - A generalization of a pointer
 - Something that can point to elements in a container
- Iterator behavior:
 - It can move from one element to another.
 - It allows generic STL algorithms to work with different containers.
- Iterator types from the slides:
 - Input iterator: reads a sequence of values
 - Output iterator: writes a sequence of values
 - Forward iterator: reads, writes, and moves forward
 - Bidirectional iterator: forward iterator plus backward movement
 - Random iterator: moves freely in any direction

Iterator Memory Aid: "IOFBR"

- I = Input reads
- O = Output writes
- F = Forward moves forward
- B = Bidirectional moves both ways
- R = Random moves anywhere

1.8 STL Algorithms

- STL algorithms are procedures applied to containers.
- Examples from the slides:
 - Searching
 - Sorting
- STL provides about 70 generic functions.
- Algorithms rely on iterator capability.
- Some algorithms are implemented as member functions for efficiency.

1.9 Vector Code Segments

- Source note:
 - Chapter 1 names Vector as an ADT/STL example.
 - Chapter 2 gives these vector-related code examples:
 - `vec.size()` is $O(1)$
 - Traversing a vector using an iterator is $O(n)$
 - Chapter 3 says adding to the end of a linked list is equivalent to `push_back()` in STL vector and STL list.
- Understand it:
 - A vector is an STL container.
 - It supports direct size checking.

- It supports iterator traversal from `begin()` to `end()`.
- Read it:
 - `vector<int> vec;` means a vector storing integers.
 - `vec.size()` returns the number of elements.
 - `vector<int>::iterator i` declares an iterator.
 - `*i` reads the value pointed to by the iterator.
 - `i++` moves to the next element.
- Write it:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> vec;

    vec.push_back(10);
    vec.push_back(20);
    vec.push_back(30);

    cout << "Size: " << vec.size() << endl; // 0(1)

    vector<int>::iterator i = vec.begin();
    for (; i < vec.end(); i++) {           // 0(n)
        cout << *i << " ";
    }

    return 0;
}
```

- Under-the-hood exam intuition:
 - `size()` is constant time because the STL stores the size as part of the vector.
 - Traversal is linear because every element is visited once.

Chapter 2: Complexity Analysis

Learning Targets

- Understand:
 - Complexity analysis
 - Big O notation
 - Common growth rates

2.1 Computational and Asymptotic Complexity

- Algorithms are essential to data structures.
- Data structures are implemented using algorithms.
- Some algorithms are more efficient than others.
- Complexity describes efficiency based on how much data must be processed.
- Time complexity:
 - Measures how long a function takes in computational steps.
- Space complexity:
 - Measures how much memory a function uses.

2.2 Performance vs Complexity

- Performance:
 - Actual time, memory, disk, or network use when the program runs.
 - Depends on the machine, compiler, and code.
- Complexity:
 - How resource needs scale as the input size grows.
- Key point:
 - Complexity affects performance.
 - Performance does not define complexity.

2.3 Big O Notation

- Big O describes:
 - Algorithm performance
 - Algorithm complexity
 - Execution time or space as input grows
- The slides describe Big O as an upper bound:
 - It focuses on the worst-case scenario.
 - Worst-case analysis removes uncertainty.

2.4 How to Determine Big O

- Rules from the slides:
 - Ignore constants.
 - Keep the fastest-growing term.
 - Drop low-order terms.
- Examples:

- $O(2n)$ becomes $O(n)$
- $O(n^2 + n + 1000)$ becomes $O(n^2)$
- Growth order from the slides:
 - $O(1)$
 - $O(\log n)$
 - $O(n)$
 - $O(n \log n)$
 - $O(n^2)$
 - $O(2^n)$
 - $O(n!)$

Big O Memory Aid: "Drop, Dominate, Decide"

- Drop constants.
- Dominate with the fastest-growing term.
- Decide the final Big O.

2.5 $O(1)$: Constant Time

- Constant time means:
 - Same amount of work regardless of input size.
 - Execution time is independent of input size.
- Slide example:
 - `vec.size()` is $O(1)$ because the STL stores the vector size.
- Read it:
 - Look for direct access or fixed-count operations.
- Write it:

```
int getVectorSize(const vector<int>& vec) {
    return vec.size(); //  $O(1)$ 
}
```

2.6 $O(n)$: Linear Time

- Linear time means:
 - Runtime grows directly with input size.
 - If input doubles, operations roughly double.
- Slide example:

```
vector<int>::iterator i = vec.begin();
for (; i < vec.end(); i++)
    cout << *i;
```

- Understand it:
 - The loop visits each element once.
- Read it:
 - One loop over n elements usually means $O(n)$.
- Write it:

```
void printVector(const vector<int>& vec) {
    for (auto i = vec.begin(); i < vec.end(); i++) {
        cout << *i << " ";
    }
}
```

2.7 $O(n^2)$: Quadratic Time

- From the slides:
 - Nested loops where each loop runs n times produce $n * n$ operations.
 - Complexity is $O(n^2)$.
- Read it:
 - A loop inside another loop over the same input is the common sign.
- Write it:

```
void printPairs(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << arr[i] << ", " << arr[j] << endl;
        }
    }
}
```

2.8 $O(\log n)$: Logarithmic Time

- Logarithmic time grows slowly as n grows.
- Slide rule:
 - A loop is often $O(\log n)$ if the counter doubles instead of increasing by 1.
- Slide correction:
 - The slide shows `for (int i = 0; i < n; i *= 2)`.
 - Starting at `i = 0` causes `i` to stay 0 forever.
 - Exam-safe corrected version starts at `i = 1`.
- Write it:

```
void logLoop(int n) {
    for (int i = 1; i < n; i *= 2) {
        cout << i << " ";
    }
}
```

2.9 $O(n \log n)$: Linearithmic Time

- From the slides:
 - Linearithmic algorithms perform well with large data sets.
 - Examples named:
 - Heapsort
 - Merge sort
 - Quick sort
- Read it:
 - Often appears as:
 - One loop over n items
 - Inside it, a logarithmic loop
- Write it:

```
void nLogNLoop(int n) {
    for (int i = 0; i < n; i++) {
        for (int j = 1; j < n; j *= 2) {
            cout << i << ", " << j << endl;
        }
    }
}
```

Chapter 3: Linked Lists, Part 1

Learning Targets

- Understand:
 - Singly linked list implementation
 - Doubly linked list implementation

3.1 Why Linked Lists?

- Arrays have two major limitations from the slides:
 - The size must be known at compile time.
 - Insertion may require shifting many elements because array elements are the same distance apart in memory.
- Linked lists help overcome these limitations.

3.2 What Is a Linked List?

- A linked list is:
 - A non-sequential collection of independent memory locations called nodes
 - A structure where each node stores data and links to other nodes
- Important idea:
 - Nodes are not necessarily stored next to each other in memory.
 - Each node points to the next node.

3.3 Advantages of Linked Lists

- Dynamic data structure.
- Efficient memory utilization:
 - Memory is allocated when needed.
 - Memory is deallocated when no longer needed.
- Insertion and deletion are easier and efficient at specified positions.

3.4 Limitations of Linked Lists

- More space is used because pointers must be stored.
- Searching can be difficult and time consuming.

3.5 Array vs Linked List

- Static array:
 - Efficient random access
 - Inefficient insertion/deletion
 - No extra pointer overhead
- Linked list:
 - Efficient insertion/deletion
 - Efficient element rearrangement
 - Inefficient random access
 - Requires 1 or 2 links per element

3.6 Types of Linked Lists

- Covered:
 - Singly linked list
 - Doubly linked list
- Mentioned but not covered in syllabus:
 - Circular linked list
 - Circular double linked list

3.7 Singly Linked List

- A singly linked list:
 - Links nodes using one pointer.
 - Moves forward in sequential order.
 - Is also called a linear linked list.
- Key pointers:
 - head stores the address of the first node.
 - Last node has next = NULL.
 - tail is optional.

3.8 Basic Singly Linked List Operations

- Creation
- Insertion
- Traversal
- Deletion

3.9 Singly Linked List Node Creation

- Understand it:
 - A node has data and a next pointer.
 - New nodes are allocated dynamically.
- Read it:
 - Node *n = new Node(); creates a node.
 - n->no = 6; stores data.
 - n->next = NULL; marks no next node yet.
- Write it:

```
class Node {
public:
    int no;
    Node *next;
};

Node* createNode(int value) {
    Node *n = new Node();
    n->no = value;
    n->next = NULL;
    return n;
}
```

Singly Linked List Memory Aid: "DAN"

- D = Data
- A = Address
- N = NULL at the end

3.10 Singly Linked List Insertion: Empty List

- Slide logic:
 - Create node.
 - Set node data.
 - Set next to NULL.
 - If `head == NULL`, assign `head = n`.
- Step-by-step:
 1. Create a new node.
 2. Store the value.
 3. Set `n->next = NULL`.
 4. Check whether the list is empty.
 5. If empty, set `head = n`.
- Write it:

```
void insertEmpty(Node *&head, int value) {
    Node *n = createNode(value);
    if (head == NULL) {
        head = n;
    }
}
```

3.11 Singly Linked List Insertion: End

- Slide logic:
 - Use pointer `p` to move to the last node.
 - Link the last node to the new node.
- Step-by-step:
 1. Create a new node.
 2. If the list is empty, set `head = n`.
 3. Otherwise, set `p = head`.
 4. Move `p` while `p->next != NULL`.
 5. Set `p->next = n`.
- Write it:

```
void insertEnd(Node *&head, int value) {
    Node *n = createNode(value);

    if (head == NULL) {
        head = n;
        return;
    }

    Node *p = head;
    while (p->next != NULL) {
        p = p->next;
    }

    p->next = n;
}
```

- STL comparison from slides:
 - Equivalent idea to `push_back()` in STL vector and STL list.

3.12 Singly Linked List Insertion: Beginning

- Slide logic:
 - Point new node to current head.

- Move head to new node.

- Step-by-step:

1. Create a new node.
2. Set `n->next = head`.
3. Set `head = n`.

- Write it:

```
void insertBeginning(Node *&head, int value) {
    Node *n = createNode(value);
    n->next = head;
    head = n;
}
```

- STL comparison from slides:

- Equivalent idea to `push_front()` in STL list.

3.13 Singly Linked List Insertion: Middle

- Slide logic:

- Move pointer `p` to one position before the insertion location.
- Connect the new node to the next node.
- Connect `p` to the new node.

- Step-by-step:

1. Create a new node.
2. Set `p = head`.
3. Move `p` until it is one node before the target position.
4. Set `n->next = p->next`.
5. Set `p->next = n`.

- Write it:

```
void insertAfterPosition(Node *head, int posBefore, int value) {
    Node *n = createNode(value);
    Node *p = head;
    int i = 1;

    while (i < posBefore && p != NULL) {
        p = p->next;
        i++;
    }

    if (p == NULL) return;

    n->next = p->next;
    p->next = n;
}
```

Insertion Memory Aid: "C-M-L"

- C = Create the node
- M = Move pointer to the correct place
- L = Link in the correct order

3.14 Singly Linked List with Tail Pointer

- Tail pointer:

- Points to the last node.
- Makes insertion at the end faster.

- Empty list insertion:

```
void insertFirstWithTail(Node *&head, Node *&tail, int value) {
    Node *n = createNode(value);
    if (head == NULL) {
        head = tail = n;
    }
}
```

- End insertion with tail:

```
void insertEndWithTail(Node *&head, Node *&tail, int value) {
    Node *n = createNode(value);

    if (head == NULL) {
        head = tail = n;
        return;
    }

    tail->next = n;
    tail = n;
}
```

3.15 Singly Linked List Deletion: Beginning

- Slide logic:
 - Store old head in temporary pointer.
 - Move head to next node.
 - Delete old head.
- Slide-code correction:
 - The slides use `free(n)`.
 - Since nodes are created using `new`, use `delete n` in C++.
- Step-by-step:
 1. If the list is empty, stop.
 2. Set `n = head`.
 3. Set `head = head->next`.
 4. Delete `n`.
- Write it:

```
void deleteBeginning(Node *&head) {
    if (head == NULL) return;

    Node *n = head;
    head = head->next;
    delete n;
}
```

3.16 Singly Linked List Deletion: Middle by Value

- Slide logic:
 - Move `n` to the node before the targeted node.
 - Put `x` at the target node.
 - Link around `x`.
 - Delete `x`.
- Step-by-step:
 1. If list is empty or has no next node, handle separately.
 2. Move `n` until `n->next` is the target.
 3. Set `x = n->next`.
 4. Set `n->next = x->next`.
 5. Delete `x`.

- Write it:

```
void deleteValue(Node *&head, int target) {
    if (head == NULL) return;

    if (head->no == target) {
        deleteBeginning(head);
        return;
    }

    Node *n = head;
    while (n->next != NULL && n->next->no != target) {
        n = n->next;
    }

    if (n->next == NULL) return;

    Node *x = n->next;
    n->next = x->next;
    delete x;
}
```

3.17 Singly Linked List Deletion: End

- Slide logic:
 - Move pointer to one node before the last node.
 - Delete last node.
 - Set second-last node's next to NULL.
- Step-by-step:
 1. If list is empty, stop.
 2. If list has one node, delete it and set head = NULL.
 3. Move n while n->next->next != NULL.
 4. Set x = n->next.
 5. Set n->next = NULL.
 6. Delete x.

- Write it:

```
void deleteEnd(Node *&head) {
    if (head == NULL) return;

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    Node *n = head;
    while (n->next->next != NULL) {
        n = n->next;
    }

    Node *x = n->next;
    n->next = NULL;
    delete x;
}
```

3.18 Deletion with Tail Pointer

- Beginning deletion with tail:
 - If deleting the only node, both head and tail become NULL.

```
void deleteBeginningWithTail(Node *&head, Node *&tail) {
    if (head == NULL) return;

    Node *n = head;
    head = head->next;
    delete n;

    if (head == NULL) {
```

```

        tail = NULL;
    }
}

```

- End deletion with tail:

```

void deleteEndWithTail(Node *&head, Node *&tail) {
    if (head == NULL) return;

    if (head == tail) {
        delete head;
        head = tail = NULL;
        return;
    }

    Node *n = head;
    while (n->next != tail) {
        n = n->next;
    }

    delete tail;
    tail = n;
    tail->next = NULL;
}

```

3.19 Singly Linked List Traversal

- Slide logic:
 - Create temporary pointer `n = head`.
 - Keep moving until `n == NULL`.
 - Print data at each node.

- Step-by-step:

1. Set `n = head`.
2. While `n != NULL`, process `n->no`;
3. Move `n = n->next`.

- Write it:

```

void display(Node *head) {
    Node *n = head;
    while (n != NULL) {
        cout << n->no << " ";
        n = n->next;
    }
}

```

3.20 Singly Linked List Update

- Based on the slide exercise:
 - Update a node containing value 7 to value 3.
- Step-by-step:
 1. Start at `head`.
 2. Traverse until value 7 is found.
 3. Change it to 3.
 4. Stop or continue depending on whether only one node should change.

- Write it:

```

void updateValue(Node *head, int oldValue, int newValue) {
    Node *n = head;
    while (n != NULL) {
        if (n->no == oldValue) {
            n->no = newValue;
            return;
        }
        n = n->next;
    }
}

```

3.21 Singly Linked List Reversal

- Source note:
 - Reversal is not directly shown in the slides.
 - It is included because the requested highlighted subchapter asks for reversing linked lists.
 - It uses the same pointer-linking ideas from insertion, deletion, and traversal.
- Understand it:
 - Reverse every `next` pointer.
 - The old head becomes the tail.
 - The old tail becomes the head.
- Read it:
 - Look for three pointers:
 - `prev`
 - `curr`
 - `nextNode`
 - The core line is `curr->next = prev`.
- Step-by-step:
 1. Set `prev = NULL`.
 2. Set `curr = head`.
 3. Save next node as `nextNode = curr->next`.
 4. Reverse current link: `curr->next = prev`.
 5. Move `prev = curr`.
 6. Move `curr = nextNode`.
 7. Repeat until `curr == NULL`.
 8. Set `head = prev`.

- Write it:

```
void reverseSingly(Node *&head) {
    Node *prev = NULL;
    Node *curr = head;

    while (curr != NULL) {
        Node *nextNode = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextNode;
    }

    head = prev;
}
```

Reversal Memory Aid: "Save, Swing, Step"

- Save the next node.
- Swing the link backward.
- Step both pointers forward.

Chapter 3: Linked Lists, Part 2

3.22 Doubly Linked List

- A doubly linked list:
 - Uses two links/pointers.
 - Can access the successor and predecessor of a node.
- Each node contains:
 - Data
 - prev link
 - next link
- prev points to predecessor.
- next points to successor.

3.23 Doubly Linked List Node Creation

- Understand it:
 - Each node must maintain two directions.
- Read it:
 - Node *next, *prev; means two node pointers.
 - Both should be initialized to NULL.
- Write it:

```
class DNode {
public:
    int no;
    DNode *next;
    DNode *prev;
};

DNode* createdNode(int value) {
    DNode *n = new DNode();
    n->no = value;
    n->next = NULL;
    n->prev = NULL;
    return n;
}
```

3.24 Doubly Linked List Insertion: Empty List

- Step-by-step:
 1. Create a new node.
 2. Set n->next = NULL.
 3. Set n->prev = NULL.
 4. If head == NULL, set head = n.
- Write it:

```
void insertEmpty(DNode *&head, int value) {
    DNode *n = createdNode(value);
    if (head == NULL) {
        head = n;
    }
}
```

3.25 Doubly Linked List Insertion: End

- Step-by-step:
 1. Create a new node.

2. If empty, set `head = n`.
3. Move `p` to the last node.
4. Set `p->next = n`.
5. Set `n->prev = p`.

- Write it:

```
void insertEnd(DNode *&head, int value) {
    DNode *n = createDNode(value);

    if (head == NULL) {
        head = n;
        return;
    }

    DNode *p = head;
    while (p->next != NULL) {
        p = p->next;
    }

    p->next = n;
    n->prev = p;
}
```

3.26 Doubly Linked List Insertion: Beginning

- Step-by-step:

1. Create a new node.
2. Set `n->next = head`.
3. Set `head->prev = n`.
4. Set `head = n`.

- Write it:

```
void insertBeginning(DNode *&head, int value) {
    DNode *n = createDNode(value);

    if (head != NULL) {
        n->next = head;
        head->prev = n;
    }

    head = n;
}
```

3.27 Doubly Linked List Insertion: Middle

- Slide logic:

- Move `p` to one node before the insertion location.
- Connect four links.

- Step-by-step:

1. Create a new node.
2. Move `p` to one position before the insertion position.
3. Set `n->next = p->next`.
4. Set `p->next->prev = n`.
5. Set `p->next = n`.
6. Set `n->prev = p`.

- Write it:

```
void insertAfter(DNode *p, int value) {
    if (p == NULL) return;

    DNode *n = createDNode(value);
    n->next = p->next;
```

```

n->prev = p;

if (p->next != NULL) {
    p->next->prev = n;
}

p->next = n;
}

```

Doubly Insertion Memory Aid: "Next Before Prev"

- First attach the new node to its future neighbors.
- Then attach the neighbors back to the new node.
- This avoids losing access to the rest of the list.

3.28 Doubly Linked List Deletion: Beginning

- Slide logic:
 - Save old head.
 - Move head to next.
 - Delete old head.
 - Set new head's `prev` to `NULL`.
- Step-by-step:
 1. If empty, stop.
 2. Set `n = head`.
 3. Set `head = head->next`.
 4. If new head exists, set `head->prev = NULL`.
 5. Delete `n`.
- Write it:

```

void deleteBeginning(DNode *&head) {
    if (head == NULL) return;

    DNode *n = head;
    head = head->next;

    if (head != NULL) {
        head->prev = NULL;
    }

    delete n;
}

```

3.29 Doubly Linked List Deletion: Middle

- Option 1 from slides:
 - Use pointer before the target.
 - Use pointer at the target.
- Option 2 from slides:
 - Move directly to the target.
 - Use `prev` and `next` links.
- Slide-code correction:
 - The slide's option 2 says `free(x)` even though the pointer shown is `n`.
 - Correct deletion should delete the targeted pointer.
- Step-by-step:
 1. Move `n` to the target node.

2. Link previous node to next node: `n->prev->next = n->next;`
3. Link next node to previous node: `n->next->prev = n->prev;`
4. Delete `n`.

- Write it:

```
void deleteMiddleNode(DNode *n) {
    if (n == NULL) return;
    if (n->prev == NULL || n->next == NULL) return;

    n->prev->next = n->next;
    n->next->prev = n->prev;
    delete n;
}
```

3.30 Doubly Linked List Deletion: End

- Step-by-step:

1. If empty, stop.
2. Move `n` until `n->next == NULL`.
3. Set `n->prev->next = NULL`.
4. Delete `n`.

- Write it:

```
void deleteEnd(DNode *&head) {
    if (head == NULL) return;

    if (head->next == NULL) {
        delete head;
        head = NULL;
        return;
    }

    DNode *n = head;
    while (n->next != NULL) {
        n = n->next;
    }

    n->prev->next = NULL;
    delete n;
}
```

3.31 Doubly Linked List Traversal

- Forward traversal:

```
void displayForward(DNode *head) {
    DNode *p = head;
    while (p != NULL) {
        cout << p->no << " ";
        p = p->next;
    }
}
```

- Backward traversal:

```
void displayBackward(DNode *tail) {
    DNode *p = tail;
    while (p != NULL) {
        cout << p->no << " ";
        p = p->prev;
    }
}
```

3.32 Doubly Linked List Reversal

- Source note:

- Reversal is included as requested.
- It extends the slide concept that DLL nodes maintain both `next` and `prev`.

- Understand it:

- Swap `next` and `prev` for every node.
 - The old last node becomes the new head.
- Step-by-step:
1. Start at `head`.
 2. For each node, save `next`.
 3. Swap the node's `next` and `prev`.
 4. Move to the saved original next node.
 5. Track the last processed node.
 6. Set `head` to the last processed node.
- Write it:

```
void reverseDoubly(DNode *&head) {  
    DNode *curr = head;  
    DNode *last = NULL;  
  
    while (curr != NULL) {  
        DNode *nextNode = curr->next;  
        curr->next = curr->prev;  
        curr->prev = nextNode;  
        last = curr;  
        curr = nextNode;  
    }  
  
    head = last;  
}
```

3.33 Doubly Linked List Advantages

- Can be traversed forward and backward.
- Deletion is more efficient if the pointer to the node is already given.
- Can insert before a given node quickly.

3.34 Doubly Linked List Disadvantages

- Extra space is needed for the previous pointer.
- Operations must maintain the extra previous pointer.

Chapter 4: Stacks

Learning Targets

- Understand:
 - Stack using STL
 - Stack using linked list
 - Stack applications

4.1 What Is a Stack?

- A stack is:
 - A restricted access linear data structure.
 - Accessible at one end only for adding/removing data.
 - LIFO: Last In First Out.
- Useful when:
 - Data must be stored and processed in reverse order.

4.2 Stack Implementation Methods

- From the slides:
 - Array-based, not covered in syllabus
 - STL
 - Linked list

4.3 STL Stack

- Main STL stack functions:
 - `empty()`: checks if stack is empty
 - `push(e1)`: pushes item onto top
 - `pop()`: removes top element
 - `top()`: returns top element without removing it
 - `swap()`: swaps two stacks of same type
 - `emplace()`: inserts new element on top
- Read it:
 - `stack<T> mystack;` creates a stack of characters.
 - `mystack.push('C');` puts C on top.
 - `mystack.top();` reads the top.
 - `mystack.pop();` removes the top.
- Write it:

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<char> mystack;

    mystack.push('C');
    mystack.push('A');
    mystack.emplace('D');

    cout << mystack.top() << endl;
```

```

while (!mystack.empty()) {
    cout << mystack.top() << " ";
    mystack.pop();
}

return 0;
}

```

4.4 Linked List Stack

- Option 1 from slides:
 - Push: insert at beginning of linked list.
 - Pop: delete at beginning of linked list.
- Option 2 from slides:
 - Push: insert at end of linked list.
 - Pop: delete at end of linked list.
- Exam preference:
 - Option 1 is simpler because both operations happen at head.
- Write it:

```

class StackLL {
    struct Node {
        int data;
        Node *next;
    };

    Node *topNode;

public:
    StackLL() {
        topNode = NULL;
    }

    bool empty() {
        return topNode == NULL;
    }

    void push(int value) {
        Node *n = new Node();
        n->data = value;
        n->next = topNode;
        topNode = n;
    }

    void pop() {
        if (topNode == NULL) return;
        Node *old = topNode;
        topNode = topNode->next;
        delete old;
    }

    int top() {
        return topNode->data;
    }
};

```

4.5 Stack Applications

- From the slides:
 - Evaluating expressions and parsing syntax
 - Balancing delimiters in program code
 - Converting numbers between bases
 - Processing financial data
 - Backtracking algorithms
 - Undo function

- Infix to postfix/prefix conversion
- Forward/backward browser feature
- Tower of Hanoi
- Tree traversals
- Stock span problem
- Histogram problem
- Graph algorithms such as topological sorting and strongly connected components

4.6 Balanced Parentheses Algorithm

- Slide algorithm:
 - Read each token.
 - Push opening brackets.
 - When closing bracket appears, pop and compare.
 - If mismatch, report error.
 - At the end, stack must be empty.
- Step-by-step:
 1. Create an empty stack.
 2. Read the expression from left to right.
 3. If token is (, {, or [, push it.
 4. If token is), }, or], check whether stack is empty.
 5. If stack is empty, expression is not balanced.
 6. Otherwise pop the top opening bracket.
 7. Compare whether the opening and closing brackets match.
 8. If they do not match, expression is not balanced.
 9. After reading all tokens, expression is balanced only if stack is empty.

Balanced Bracket Memory Aid: "Push Open, Pop Close, Empty Ends"

- Push every opening bracket.
- Pop every closing bracket.
- Empty stack at the end means balanced.

C++ Balanced Parentheses Function

```
#include <stack>
#include <string>
using namespace std;

bool matches(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

bool isBalanced(const string& expr) {
    stack<char> st;

    for (char token : expr) {
        if (token == '(' || token == '[' || token == '{') {
            st.push(token);
        } else if (token == ')' || token == ']' || token == '}') {
            if (st.empty()) return false;

            char open = st.top();
            st.pop();
        }
    }

    return st.empty();
}
```

```

        if (!matches(open, token)) return false;
    }
}

return st.empty();
}

```

4.7 Infix to Postfix Conversion

- Infix:
 - Human/programmer notation.
 - Example: $A * (B + C)$
- Postfix:
 - Easier for compilers to parse.
 - No parentheses required during evaluation.

Operator Priority

- Highest in the slides:
 - $*$, $/$
- Lower:
 - $+$, $-$
- Parentheses:
 - Force grouping.

Infix to Postfix Step-by-Step Algorithm

1. Create an empty stack.
2. Scan tokens from left to right.
3. If token is an operand:
 - Display/add it to postfix output.
4. If token is $($:
 - Push it onto the stack.
5. If token is $)$:
 - Pop and display stack items until $($ is found.
 - Do not display $($.
 - If $($ is not found, report an error.
6. If token is an operator:
 - If stack is empty, push it.
 - If token has higher priority than stack top, push it.
 - Otherwise, pop and display stack top.
 - Repeat comparison with remaining stack items.
 - Push current operator.
7. After the expression ends:
 - Pop and display all remaining operators.

Infix to Postfix Memory Aid: "Operand Out, Open On, Close Clears, Operators Compare"

- Operand Out: operands go straight to output.
- Open On: opening parenthesis goes onto stack.
- Close Clears: closing parenthesis clears until opening parenthesis.

- Operators Compare: compare precedence before pushing.

Detailed Dry Run 1: A * (B + C)

- Tokens:
 - A, *, (, B, +, C,)
- Initial:
 - Stack: empty
 - Output: empty

Step	Token	Action	Stack	Output
1	A	Operand goes to output	empty	A
2	*	Stack empty, push operator	*	A
3	(	Push opening parenthesis	* (	A
4	B	Operand goes to output	* (	A B
5	+	Top is (, push operator	* (+	A B
6	C	Operand goes to output	* (+	A B C
7	)	Pop until (, output +, discard (	*	A B C +
End	none	Pop remaining *	empty	A B C + *

- Final postfix:
 - A B C + *

Detailed Dry Run 2: 7 * 8 - (2 + 3)

- Tokens:
 - 7, *, 8, -, (, 2, +, 3,)
- Initial:
 - Stack: empty
 - Output: empty

Step	Token	Action	Stack	Output
1	7	Operand to output	empty	7
2	*	Push operator	*	7
3	8	Operand to output	*	7 8
4	-	* has higher priority, pop *; push -	-	7 8 *
5	(	Push opening parenthesis	- (	7 8 *
6	2	Operand to output	- (	7 8 * 2
7	+	Push after (	- (+	7 8 * 2
8	3	Operand to output	- (+	7 8 * 2 3
9	)	Pop +, discard (	-	7 8 * 2 3 +
End	none	Pop remaining -	empty	7 8 * 2 3 + -

- Final postfix:
 - 7 8 * 2 3 + -

C++ Infix to Postfix

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;
```

```

int precedence(char op) {
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
    return 0;
}

bool isOperator(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

string infixToPostfix(const string& infix) {
    stack<char> st;
    string postfix;

    for (char token : infix) {
        if (token == ' ') continue;

        if (isdigit(token)) {
            postfix += token;
            postfix += ' ';
        } else if (token == '(') {
            st.push(token);
        } else if (token == ')') {
            while (!st.empty() && st.top() != '(') {
                postfix += st.top();
                postfix += ' ';
                st.pop();
            }
            if (!st.empty()) st.pop();
        } else if (isOperator(token)) {
            while (!st.empty() && st.top() != '(' &&
                precedence(st.top()) >= precedence(token)) {
                postfix += st.top();
                postfix += ' ';
                st.pop();
            }
            st.push(token);
        }
    }

    while (!st.empty()) {
        postfix += st.top();
        postfix += ' ';
        st.pop();
    }

    return postfix;
}

```

4.8 Postfix Evaluation

- Slide algorithm:
 - Create empty stack.
 - Read token.
 - If operand, push.
 - If operator, pop top two items.
 - Execute expression.
 - Push result.
 - Final value is at top of stack.
- Step-by-step:
 1. Create empty stack.
 2. Scan postfix expression left to right.
 3. If token is operand, push it.
 4. If token is operator, pop right operand.
 5. Pop left operand.
 6. Calculate left operator right.

7. Push result.

8. At the end, stack top is the final answer.

- Slide example:

- 2 4 * 9 5 + -

- 2 4 * = 8

- 9 5 + = 14

- 8 14 - = -6

- Write it:

```
int evalPostfix(const vector<string>& tokens) {
    stack<int> st;

    for (string token : tokens) {
        if (token == "+" || token == "-" || token == "*" || token == "/") {
            int right = st.top(); st.pop();
            int left = st.top(); st.pop();

            if (token == "+") st.push(left + right);
            else if (token == "-") st.push(left - right);
            else if (token == "*") st.push(left * right);
            else st.push(left / right);
        } else {
            st.push(stoi(token));
        }
    }

    return st.top();
}
```

Chapter 5: Queues

Learning Targets

- Understand:
 - Queue using STL
 - Queue using linked list
 - Queue applications

5.1 What Is a Queue?

- A queue is:
 - A restricted access linear data structure.
 - Insertions happen at the rear/end.
 - Deletions happen at the front.
 - FIFO: First In First Out.
- Queue operations:
 - Enqueue/append: insert at back.
 - Dequeue/serve: remove from front.

5.2 STL Queue

- Main functions:
 - `empty()`: checks whether queue is empty
 - `push(el)`: inserts at the end
 - `pop()`: removes first element
 - `front()`: returns first element
 - `back()`: returns last element
 - `swap()`: swaps queues of same type
 - `emplace()`: inserts new element at the end
- Write it:

```
#include <iostream>
#include <queue>
using namespace std;

int main() {
    queue<char> myQ;

    myQ.push('C');
    myQ.push('A');
    myQ.emplace('D');

    cout << "Front: " << myQ.front() << endl;
    cout << "Back: " << myQ.back() << endl;

    while (!myQ.empty()) {
        cout << myQ.front() << " ";
        myQ.pop();
    }

    return 0;
}
```

5.3 Linked List Queue

- Option 1 from slides:
 - Enqueue: insert at end.

- Dequeue: remove at beginning.
- Option 2 from slides:
 - Enqueue: insert at beginning.
 - Dequeue: remove at end.
- Exam preference:
 - Option 1 with `head` and `tail` is natural and efficient.

Queue Memory Aid: "Back In, Front Out"

- Back In = enqueue at rear.
- Front Out = dequeue at front.

C++ Linked List Queue

```
class QueueLL {
    struct Node {
        int data;
        Node *next;
    };

    Node *frontNode;
    Node *rearNode;

public:
    QueueLL() {
        frontNode = rearNode = NULL;
    }

    bool empty() {
        return frontNode == NULL;
    }

    void enqueue(int value) {
        Node *n = new Node();
        n->data = value;
        n->next = NULL;

        if (rearNode == NULL) {
            frontNode = rearNode = n;
        } else {
            rearNode->next = n;
            rearNode = n;
        }
    }

    void dequeue() {
        if (frontNode == NULL) return;

        Node *old = frontNode;
        frontNode = frontNode->next;
        delete old;

        if (frontNode == NULL) {
            rearNode = NULL;
        }
    }

    int front() {
        return frontNode->data;
    }
};
```

5.4 Queue Applications

- From the slides:
 - CPU scheduling
 - Disk scheduling
 - IO buffers
 - Pipes

- File IO
- Printer spooling
- Website traffic
- Media player playlist
- Network interrupts
- Routers and switches
- Mail systems

Chapter 6: Tree, Part 1

Learning Targets

- Understand:
 - Tree fundamentals
 - Binary tree
 - Binary search tree
 - Expression tree
 - Tree traversal
 - Tree applications

6.1 What Is a Tree?

- A tree is:
 - Non-linear
 - Hierarchical
 - A collection of nodes
 - Nodes store a value and references to children
 - Connected through edges
- The slides also describe Tree as:
 - A widely used ADT that simulates hierarchy.

6.2 Types of Tree Data Structures

- General tree:
 - No restriction on number of children.
- Binary tree:
 - Each node has at most 2 children.
 - Children are usually named left and right.
- Binary search tree:
 - Left node value is less than parent.
 - Right node value is greater than parent.
- Expression tree:
 - Internal nodes are operators.
 - Leaf nodes are operands.

6.3 Tree Terminology

- Root:
 - Node with no parent.
 - Topmost node.
- Edge:
 - Line connecting two nodes.
 - Also known as branch.
- Children:

- Immediate successor nodes.
- Siblings:
 - Children of the same parent.
- Leaf node:
 - Node with no children.
 - Also called external or terminal node.
- Internal node:
 - Node with at least one child.
- Size:
 - Number of nodes.
- Depth of node x:
 - Length of path from root to node x.
- Height of node x:
 - Number of edges in the longest path from node x to a leaf.
- Height of tree:
 - Height of root.
- Level:
 - Set of nodes at a given depth.

Tree Terminology Memory Aid: "R-E-C-S-L-I"

- R = Root
- E = Edge
- C = Children
- S = Siblings
- L = Leaf
- I = Internal

6.4 Binary Tree

- A binary tree:
 - Has at most two children per node.
 - Is the simplest and most common tree type in the slides.
- Types from the slides:
 - Rooted binary tree
 - Degenerate binary tree
 - Full binary tree
 - Complete binary tree

6.5 Binary Tree Types and Formulas

- Binary tree:
 - Minimum nodes for height h : $h + 1$
 - Maximum nodes for height h : $2^{(h+1)} - 1$
 - Minimum height for n nodes: $\lceil \log(n + 1) \rceil - 1$
 - Maximum height for n nodes: $n - 1$

- Full binary tree:
 - Every node has either 0 or 2 children.
 - Number of leaf nodes = number of internal nodes + 1.
 - Minimum nodes for height h : $2h + 1$
 - Maximum nodes for height h : $2^{(h+1)} - 1$
 - Maximum height for n nodes: $(n - 1) / 2$
- Complete binary tree:
 - All levels are filled except possibly the last.
 - Last level nodes are as left as possible.
 - Minimum nodes for height h : 2^h
 - Maximum nodes for height h : $2^{(h+1)} - 1$
 - Maximum height for n nodes: $\log(n)$

6.6 Binary Search Tree

- A BST is an enhanced binary tree.
- It needs:
 - A key
 - A way to compare keys
 - A storage rule
- BST rules:
 - Left child key is less than parent key.
 - Right child key is greater than or equal to parent key.
 - Left and right children are also BSTs.
- Performance from slides:
 - Searching improves to $O(\log n)$ by eliminating about 50% of potential nodes.

6.7 BST Insertion

- Slide algorithm:
 - If tree is empty, create root.
 - Otherwise compare with current node.
 - If equal, replace value.
 - If greater, go right.
 - If smaller, go left.
 - Insert when an empty subtree is found.
- Step-by-step:
 1. If `root == NULL`, new node becomes root.
 2. Compare new key with current node.
 3. If new key is smaller, move left.
 4. If new key is greater or equal, move right.
 5. Repeat until the correct empty child pointer is found.
 6. Attach the new node there.
- Write it:

```

struct TreeNode {
    int data;
    TreeNode *left;
    TreeNode *right;
};

TreeNode* createTreeNode(int value) {
    TreeNode *n = new TreeNode();
    n->data = value;
    n->left = n->right = NULL;
    return n;
}

void insert(TreeNode *&root, int value) {
    if (root == NULL) {
        root = createTreeNode(value);
    } else if (value < root->data) {
        insert(root->left, value);
    } else {
        insert(root->right, value);
    }
}

```

BST Insertion Memory Aid: "Compare, Choose, Continue, Connect"

- Compare with current node.
- Choose left or right.
- Continue down the tree.
- Connect at empty spot.

6.8 BST Deletion

- Three conditions from the slides:
 - Condition 1: node is a leaf node.
 - Condition 2: node has one child.
 - Condition 3: node has two children.

Condition 1: Leaf Node

- Step:
 - Delete the node directly.

Condition 2: One Child

- Steps:
 1. Change the parent's reference to point to the child of the deleted node.
 2. The child and its subtrees take the deleted node's place.

Condition 3: Two Children

- Steps from the slides:
 1. Replace the node with its inorder successor.
 2. The inorder successor is the last left node in the right subtree.
 3. The inorder successor cannot have a left child.
 4. If it has a right child, that child occupies the successor's old position.

C++ BST Deletion

```

TreeNode* findMin(TreeNode *root) {
    while (root != NULL && root->left != NULL) {
        root = root->left;
    }
    return root;
}

TreeNode* deleteBST(TreeNode *root, int key) {

```

```

if (root == NULL) return NULL;

if (key < root->data) {
    root->left = deleteBST(root->left, key);
} else if (key > root->data) {
    root->right = deleteBST(root->right, key);
} else {
    if (root->left == NULL && root->right == NULL) {
        delete root;
        return NULL;
    } else if (root->left == NULL) {
        TreeNode *child = root->right;
        delete root;
        return child;
    } else if (root->right == NULL) {
        TreeNode *child = root->left;
        delete root;
        return child;
    } else {
        TreeNode *successor = findMin(root->right);
        root->data = successor->data;
        root->right = deleteBST(root->right, successor->data);
    }
}

return root;
}

```

BST Deletion Memory Aid: "Zero, One, Two"

- Zero children: delete directly.
- One child: child replaces node.
- Two children: successor replaces node.

Chapter 6: Tree, Part 2

6.9 Expression Tree

- Expression tree:
 - A binary tree.
 - Internal nodes are arithmetic operators.
 - Leaf nodes are operands.
- Example from slides:
 - `A * (B + C)`
 - Root is `*`
 - Left child is `A`
 - Right subtree is `+` with children `B` and `C`

6.10 Tree Traversal

- Two main traversal approaches:
 - Breadth First Traversal / Level Order Traversal
 - Depth First Traversals
- DFS types:
 - Inorder: Left, Root, Right
 - Preorder: Root, Left, Right
 - Postorder: Left, Right, Root

6.11 BFS: Breadth First Traversal

- From slides:
 - BFS uses a queue.
 - Starts from root.
 - Visits and marks one node at a time.
 - Stores adjacent nodes in the queue.
 - Suitable for nodes closer to root.
- Step-by-step:
 1. Create an empty queue.
 2. Put root in queue.
 3. While queue is not empty, remove front node.
 4. Visit it.
 5. Add its left child if present.
 6. Add its right child if present.
- Write it:

```
#include <queue>

void bfs(TreeNode *root) {
    if (root == NULL) return;

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode *p = q.front();
```

```

        q.pop();

        cout << p->data << " ";

        if (p->left != NULL) q.push(p->left);
        if (p->right != NULL) q.push(p->right);
    }
}

```

6.12 DFS Traversals

- From slides:
 - DFS is edge-based.
 - DFS may traverse more edges to reach a destination.
 - Suitable when solutions are away from source.
 - Suitable for games or puzzles.

Preorder: Root, Left, Right

- Step-by-step:
 1. Visit root.
 2. Traverse left subtree.
 3. Traverse right subtree.
- Write it:

```

void preorder(TreeNode *p) {
    if (p != NULL) {
        cout << p->data << " ";
        preorder(p->left);
        preorder(p->right);
    }
}

```

Inorder: Left, Root, Right

- Step-by-step:
 1. Traverse left subtree.
 2. Visit root.
 3. Traverse right subtree.
- Write it:

```

void inorder(TreeNode *p) {
    if (p != NULL) {
        inorder(p->left);
        cout << p->data << " ";
        inorder(p->right);
    }
}

```

Postorder: Left, Right, Root

- Step-by-step:
 1. Traverse left subtree.
 2. Traverse right subtree.
 3. Visit root.
- Write it:

```

void postorder(TreeNode *p) {
    if (p != NULL) {
        postorder(p->left);
        postorder(p->right);
        cout << p->data << " ";
    }
}

```

Traversal Memory Aid: "V Position"

- Preorder: VLR = Visit first.
- Inorder: LVR = Visit in middle.
- Postorder: LRV = Visit last.

6.13 Binary Postfix: Postorder on Expression Tree

- Source connection:
 - Slides cover expression trees and postorder traversal.
 - A postfix expression matches postorder traversal of an expression tree.
- Understand it:
 - Operators are internal nodes.
 - Operands are leaves.
 - Postorder prints left operand/subexpression, right operand/subexpression, then operator.
- Example:
 - Expression tree for $A * (B + C)$
 - Postorder:
 - Left: A
 - Right subtree: B C +
 - Root: *
 - Binary postfix result:
 - A B C + *
 - C++ expression tree node:

```
struct ExprNode {
    char value;
    ExprNode *left;
    ExprNode *right;
};

void printPostfix(ExprNode *root) {
    if (root == NULL) return;

    printPostfix(root->left);
    printPostfix(root->right);
    cout << root->value << " ";
}
```

6.14 Tree Linked List Implementation

- Tree node from slides:
 - Data
 - Left pointer
 - Right pointer
- Write it:

```
class Node {
public:
    int data;
    Node *left;
    Node *right;
};
```

- Node creation:

```
Node* createNode(int value) {
    Node *n = new Node();
    n->data = value;
    n->left = n->right = NULL;
}
```

```
    return n;
}
```

- BST insertion from slide sample:

```
void insert(Node *n, Node *&parent) {
    if (parent == NULL) {
        parent = n;
    } else if (n->data < parent->data) {
        insert(n, parent->left);
    } else {
        insert(n, parent->right);
    }
}
```

6.15 BST Search

- Slide logic:
 - Start at root.
 - If value matches, found.
 - If current value is greater than key, move left.
 - Otherwise move right.

- Step-by-step:

1. Set `s = root`.
2. While `s != NULL`, compare key.
3. If `s->data == key`, found.
4. If `s->data > key`, move left.
5. Else move right.
6. If loop ends, not found.

- Write it:

```
bool searchBST(Node *root, int key) {
    Node *s = root;

    while (s != NULL) {
        if (s->data == key) return true;

        if (s->data > key) {
            s = s->left;
        } else {
            s = s->right;
        }
    }

    return false;
}
```

6.16 Tree Summary

- Trees are useful when data naturally forms a hierarchy.
- Example from slides:
 - File system folder structure.
- BST search can be faster than linked list search.
- Tree insertion and deletion can be performed in moderate time.

Chapter 7: Graph

Learning Targets

- Understand:
 - Graph fundamentals
 - Graph types
 - Graph representation
 - Shortest paths
 - Spanning tree

7.1 What Is a Graph?

- A graph is:
 - A non-linear data structure.
 - A finite set of vertices/nodes.
 - A set of edges connecting pairs of nodes.
- Simple graph notation from slides:
 - $G = (V, E)$
 - V is the set of vertices.
 - E is a collection of unordered pairs $\{u, v\}$.
- Order of graph:
 - Number of vertices, written $|V|$.
- Size of graph:
 - Number of edges, written $|E|$.

7.2 Graph Applications

- From slides:
 - City paths
 - Telephone networks
 - Circuit networks
 - Social networks
 - Web pages
 - Network connection
 - Algorithm analysis
 - Molecular structure
 - Cell connection
 - Road network/GPS
 - Organization hierarchy
 - Financial uses

7.3 Graph Types

- Finite graph:
 - Finite vertices and finite edges.
- Infinite graph:

- Infinite vertices and edges.
- Trivial graph:
 - One vertex and no edge.
- Null graph:
 - n vertices and zero edges.
- Simple graph:
 - No more than one edge between a pair of vertices.
- Directed graph/digraph:
 - Edges have direction.
- Undirected graph:
 - Edges have no direction.
- Weighted graph:
 - Edges have costs/weights.
- Multigraph:
 - May contain parallel edges but no self-loop.
- Pseudograph:
 - Has self-loop and may have multiple edges.

7.4 Graph Representation: Adjacency List

- An adjacency list contains:
 - A list of vertices.
 - For each vertex, a list of adjacent vertices.
- Can be designed as:
 - Table/star representation
 - Linked list
- Array size equals number of vertices.
- Weighted graph:
 - Store pairs such as (neighbor, weight).
- Write it:

```
#include <vector>
using namespace std;

vector<vector<int>> adjList(5);

void addEdge(int u, int v) {
    adjList[u].push_back(v);
}
```

7.5 Graph Representation: Adjacency Matrix

- An adjacency matrix:
 - Uses a 2D array of size $V \times V$.
 - Put 1 at `matrix[i][j]` if edge exists.
 - For weighted graphs, store weight w .
- Write it:

```
const int V = 5;
int adj[V][V] = {0};
```

```
void addEdgeMatrix(int u, int v) {
    adj[u][v] = 1;
}

void addWeightedEdge(int u, int v, int w) {
    adj[u][v] = w;
}
```

7.6 Dijkstra's Shortest Path Algorithm

- From slides:
 - Proposed by Edsger Dijkstra in 1959.
 - Applies to weighted directed or undirected graphs.
 - Requires non-negative edge weights.
 - Finds shortest distance/path from start node to target/all nodes.
- Slide algorithm:
 - Initialize $S = \{1\}$.
 - Initialize distances from starting vertex.
 - Repeatedly choose vertex w outside S with minimum distance.
 - Add w to S .
 - Relax neighboring distances:
 - $D[v] = \min(D[v], D[w] + C[w, v])$
- Step-by-step:
 1. Choose start vertex.
 2. Set start distance to 0.
 3. Set other distances to infinity.
 4. Mark all vertices as unvisited.
 5. Select unvisited vertex with smallest distance.
 6. Mark it as visited.
 7. For each unvisited neighbor, calculate new distance through selected vertex.
 8. Keep the smaller distance.
 9. Repeat until all vertices are visited or no reachable vertex remains.

Dijkstra Memory Aid: "Smallest, Settle, Shorten"

- Smallest distance vertex is selected.
- Settle it into the visited set.
- Shorten neighbors using relaxation.

C++ Dijkstra with Adjacency Matrix

```
#include <iostream>
#include <vector>
#include <limits>
using namespace std;

vector<int> dijkstra(vector<vector<int>>& cost, int start) {
    int n = cost.size();
    vector<int> dist(n, INT_MAX);
    vector<bool> visited(n, false);

    dist[start] = 0;

    for (int count = 0; count < n - 1; count++) {
        int w = -1;

        for (int i = 0; i < n; i++) {
            if (!visited[i] && (w == -1 || dist[i] < dist[w])) {
```

```

        w = i;
    }
}

if (w == -1 || dist[w] == INT_MAX) break;

visited[w] = true;

for (int v = 0; v < n; v++) {
    if (!visited[v] && cost[w][v] > 0 &&
        dist[w] + cost[w][v] < dist[v]) {
        dist[v] = dist[w] + cost[w][v];
    }
}
}

return dist;
}

```

7.7 Spanning Tree

- A spanning tree:
 - Is a subset of a connected graph.
 - Connects all vertices.
 - Has no cycle.
 - For N vertices, has $N - 1$ edges.
- Minimum spanning tree:
 - For weighted connected undirected graph.
 - Has weight less than or equal to every other spanning tree.
 - Weight is the sum of edge weights.

7.8 Kruskal's Algorithm

- From slides:
 - Finds minimum cost spanning tree.
 - Uses greedy approach.
 - Picks the smallest edge that does not cause a cycle.
- Step-by-step:
 1. Sort all edges in non-decreasing order of weight.
 2. Pick the smallest edge.
 3. Check whether it forms a cycle.
 4. If no cycle, include it.
 5. If cycle, discard it.
 6. Repeat until spanning tree has $V - 1$ edges.

Kruskal Memory Aid: "Sort, Select, Skip Cycles"

- Sort edges.
- Select smallest valid edge.
- Skip edges that create cycles.

C++ Kruskal

```

#include <algorithm>
#include <vector>
using namespace std;

struct Edge {
    int u;
    int v;

```

```

    int w;
};

int findParent(vector<int>& parent, int x) {
    if (parent[x] == x) return x;
    parent[x] = findParent(parent, parent[x]);
    return parent[x];
}

bool unite(vector<int>& parent, int a, int b) {
    int pa = findParent(parent, a);
    int pb = findParent(parent, b);

    if (pa == pb) return false;

    parent[pb] = pa;
    return true;
}

int kruskal(int V, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), [](Edge a, Edge b) {
        return a.w < b.w;
    });

    vector<int> parent(V);
    for (int i = 0; i < V; i++) parent[i] = i;

    int total = 0;
    int edgeCount = 0;

    for (Edge e : edges) {
        if (unite(parent, e.u, e.v)) {
            total += e.w;
            edgeCount++;
            if (edgeCount == V - 1) break;
        }
    }

    return total;
}

```

Chapter 8: Sorting

Learning Targets

- Understand:
 - Sorting algorithm types
 - Sorting implementation

8.1 What Is Sorting?

- Sorting means arranging data in a particular format.
- Sorting improves searching efficiency.
- Choice of sorting method depends on:
 - Application
 - User needs
 - Number of comparisons
 - Number of data movements
 - Complexity
 - Practical data size

8.2 Insertion Sort

- From slides:
 - Builds final sorted list one item at a time.
 - Each repetition picks an input element.
 - Inserts it into the correct position in the sorted part.
 - Array is virtually split into sorted and unsorted parts.
- Step-by-step ascending algorithm:
 1. Start from `arr[1]`.
 2. Treat elements before it as sorted.
 3. Store current element as `key`.
 4. Compare `key` with its predecessor.
 5. While previous elements are greater than `key`, move them one position right.
 6. Insert `key` into the empty position.
 7. Repeat until all elements are processed.
- Time complexity:
 - Best case: $O(n)$ when already sorted.
 - Average/worst case: $O(n^2)$.
 - Note: The slides focus on mechanics; these complexity details complete the algorithm explanation.

Insertion Sort Memory Aid: "Pick, Shift, Place"

- Pick the key.
- Shift larger elements.
- Place key in the hole.

C++ Insertion Sort

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }
}
```

- Read it:
 - Outer loop selects the next unsorted item.
 - `key` is the item being inserted.
 - Inner loop shifts larger values right.
 - `arr[j + 1] = key` fills the open slot.

8.3 Selection Sort

- From slides:
 - Repeatedly finds the minimum element from unsorted part.
 - Places it at the beginning.
 - Maintains sorted and unsorted subarrays.
- Step-by-step:
 1. Set the first unsorted index as `i`.
 2. Find the smallest item from `i` to end.
 3. Swap smallest item with item at `i`.
 4. Move `i` one step right.
 5. Repeat until sorted.
- Write it:

```
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        swap(arr[i], arr[minIndex]);
    }
}
```

Selection Sort Memory Aid: "Find Min, Front It"

- Find the minimum.
- Move it to the front of the unsorted part.

8.4 Bubble Sort

- From slides:
 - Repeatedly steps through the list.
 - Compares adjacent pairs.
 - Swaps if in wrong order.
 - Repeats until no swaps are needed.

- Smaller elements "bubble" to the top.
- Step-by-step:
 1. Compare adjacent elements.
 2. Swap if left element is greater than right element.
 3. Continue to the end of the current pass.
 4. Reduce the unsorted range.
 5. Repeat until no swaps are needed.
- Write it:

```
void bubbleSort(int arr[], int n) {
    bool swapped;

    do {
        swapped = false;
        for (int i = 0; i < n - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                swap(arr[i], arr[i + 1]);
                swapped = true;
            }
        }
        n--;
    } while (swapped);
}
```

Bubble Sort Memory Aid: "Compare Neighbors, Swap, Shrink"

- Compare neighbors.
- Swap wrong pairs.
- Shrink the unsorted end.

8.5 Heap Sort

- From slides:
 - Comparison-based sorting.
 - Based on Binary Heap.
 - Similar to selection sort.
 - Uses min-heap or max-heap idea.
- Binary heap properties:
 - Complete tree.
 - Min heap: root is minimum.
 - Max heap: root is maximum.
- Ascending max-heap algorithm from slides:
 1. Turn original array into heap array.
 2. Exchange root/largest element with last unsorted element.
 3. Reheap down to rebuild heap.
 4. Repeat exchange and reheap until sorted.
- Write it:

```
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest]) {
        largest = left;
    }

    if (right < n && arr[right] > arr[largest]) {
```

```

        largest = right;
    }

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    for (int end = n - 1; end > 0; end--) {
        swap(arr[0], arr[end]);
        heapify(arr, end, 0);
    }
}

```

Heap Sort Memory Aid: "Heap, Swap, Reheap"

- Build heap.
- Swap root with last unsorted item.
- Reheap the remaining heap.

8.6 Quick Sort

- From slides:
 - Divide and conquer algorithm.
 - Picks a pivot.
 - Partitions array around pivot.
 - The syllabus covers choosing the last element as pivot.
- Step-by-step recursive quick sort:
 1. If `low < high`, continue.
 2. Partition the array.
 3. Pivot is placed in correct position.
 4. Recursively quick sort the left side.
 5. Recursively quick sort the right side.
- Partition step:
 1. Choose `arr[high]` as pivot.
 2. Set `i = low - 1`.
 3. For `j = low` to `high - 1`, compare `arr[j]` with pivot.
 4. If `arr[j] < pivot`, increment `i` and swap `arr[i]` with `arr[j]`.
 5. After loop, swap `arr[i + 1]` with pivot.
 6. Return `i + 1` as partition index.
- Write it:

```

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);
    return i + 1;
}

```

```

}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

```

Quick Sort Memory Aid: "Pivot, Partition, Proceed"

- Pivot is chosen.
- Partition places smaller left and larger right.
- Proceed recursively.

8.7 Merge Sort

- Source note:
 - Merge sort is named in Chapter 2 as an $O(n \log n)$ linearithmic algorithm.
 - The sorting deck does not provide detailed merge sort mechanics.
 - The code and steps here are supplemental to explain the named algorithm.
- Understand it:
 - Divide array into halves.
 - Sort each half.
 - Merge sorted halves.
- Step-by-step:
 1. If the array has one or zero elements, it is already sorted.
 2. Split the array into left and right halves.
 3. Recursively sort left half.
 4. Recursively sort right half.
 5. Merge the two sorted halves into one sorted array.
- Time complexity:
 - $O(n \log n)$, matching the Chapter 2 linearithmic category.
- Write it:

```

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

```

```
mergeSort(arr, left, mid);  
mergeSort(arr, mid + 1, right);  
merge(arr, left, mid, right);  
}
```

Merge Sort Memory Aid: "Split, Sort, Stitch"

- Split into halves.
- Sort each half.
- Stitch the sorted halves together.

Chapter 9: Searching

Learning Targets

- Understand:
 - Searching algorithm types
 - Searching implementation

9.1 What Is Searching?

- Searching is finding required information from a collection of items.
- Items may be stored in:
 - Array
 - Linked list
 - Graph
 - Tree
- Slide examples:
 - Telephone number/name lookup
 - Student/staff/sales databases
 - Internet search engines

9.2 Types of Searching

- Sequential search:
 - Traverse list/array sequentially.
 - Check every component.
 - Example: Linear search.
- Interval search:
 - Designed for sorted data structures.
 - Repeatedly targets the center.
 - Divides search space in half.
 - Example: Binary search.
- Search by hashing.

9.3 Sequential Search

- From slides:
 - Simplest search algorithm.
 - Checks each item until target is found or collection ends.
 - Preferred when data is unsorted.
- Time complexity:
 - Best case: $O(1)$, target at first element.
 - Worst case: $O(n)$, target at tail or not present.
 - Average case: target somewhere in the middle.
- Step-by-step:
 1. Start at first element.

2. Compare current element with target.
3. If equal, return found.
4. Otherwise move to next element.
5. If end is reached, return not found.

- Write it:

```
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}
```

Sequential Search Memory Aid: "Look, Match, Move"

- Look at current item.
- Match with key.
- Move if not found.

9.4 Binary Search

- From slides:
 - Works on sorted array.
 - Repeatedly divides search interval in half.
 - Uses divide and conquer.
 - Runtime complexity: $O(\log n)$.
- Important:
 - Data must be sorted first.
- Slide algorithm:
 - Mark first element as LOW.
 - Mark last element as HI.
 - Calculate $MID = (LOW + HI) / 2$.
 - If key equals middle item, found.
 - If key is smaller, move HI to $MID - 1$.
 - If key is larger, move LOW to $MID + 1$.
 - Stop when found or LOW and HI cross.
- Step-by-step:
 1. Set $low = 0$.
 2. Set $high = n - 1$.
 3. While $low \leq high$, calculate middle.
 4. Compare $arr[mid]$ with key.
 5. If equal, return mid .
 6. If key is smaller, set $high = mid - 1$.
 7. If key is larger, set $low = mid + 1$.
 8. Return not found when $low > high$.

- Write it:

```
int binarySearch(int arr[], int n, int key) {
    int low = 0;
```

```

int high = n - 1;

while (low <= high) {
    int mid = (low + high) / 2;

    if (arr[mid] == key) {
        return mid;
    } else if (key < arr[mid]) {
        high = mid - 1;
    } else {
        low = mid + 1;
    }
}

return -1;
}

```

Binary Search Memory Aid: "Low, Mid, High, Halve"

- Low and high mark the search range.
- Mid is checked.
- Halve the search space each time.

9.5 Hashing

- Hashing:
 - Maps a large amount of data into small tables using a hash function.
 - Identifies a specific item from similar items.
 - Uses hash tables to store data in array format.
- Hash function:
 - Converts an item/key into a small integer/hash value.
 - This value is used as an index.
- Example from slides:
 - `index = ID % array size`

9.6 Requirements of a Good Hash Function

- Easy to compute.
- Distributes keys evenly.
- Avoids clustering.
- Avoids collisions when possible.

9.7 Hashing Functions

Truncation Method

- Truncates part of the key depending on table size.
- Can use:
 - Rightmost digits
 - Leftmost digits
 - Middle digits
- Write it:

```

int truncateRightTwo(int key) {
    return key % 100;
}

int truncateLeftTwoOfFiveDigit(int key) {
    return key / 1000;
}

```

```
int extractMiddleTwoOfFiveDigit(int key) {  
    return (key / 100) % 100;  
}
```

Digit Extraction Method

- Select digits from the key and use them as address.
- Example from slides:
 - Key 90020479
 - Extract first, fourth, and last three digits.
 - Index becomes 92479.

Modular Arithmetic

- Formula:
 - $\text{index} = \text{key} \% \text{size}$
- Example:
 - $100252 \% 100 = 52$
- Write it:

```
int hashMod(int key, int size) {  
    return key % size;  
}
```

9.8 Collision

- A collision happens when:
 - Two keys are assigned the same index.
- Problem:
 - Each hash table index is expected to store one value.
- Collision techniques from slides:
 - Linear probing
 - Quadratic probing
 - Chaining
 - Double hashing

9.9 Linear Probing

- From slides:
 - Start at collision point.
 - Search linearly for next empty cell.
 - If end of table is reached, start from first index.
- Step-by-step:
 1. Calculate hash index.
 2. If empty, insert.
 3. If occupied, move to next index.
 4. Wrap to index 0 after table end.
 5. Repeat until empty cell is found.
- Write it:

```
void insertLinear(vector<int>& table, int key, int emptyValue) {  
    int size = table.size();  
    int index = key % size;
```

```

while (table[index] != emptyValue) {
    index = (index + 1) % size;
}

table[index] = key;
}

```

Linear Probing Memory Aid: "Hit, Step, Wrap"

- Hit a collision.
- Step to next slot.
- Wrap around if needed.

9.10 Quadratic Probing

- From slides:
 - If slot is occupied, try:
 - $(\text{hash}(x) + 1^2) \% \text{size}$
 - $(\text{hash}(x) + 2^2) \% \text{size}$
 - $(\text{hash}(x) + 3^2) \% \text{size}$
 - Repeat with increasing i .
- Formula:
 - $(\text{hash value} + i^2) \bmod \text{table_size}$
- Step-by-step:
 1. Calculate original hash index.
 2. If empty, insert.
 3. If occupied, set $i = 1$.
 4. Try $(\text{index} + i * i) \% \text{size}$.
 5. Increase i .
 6. Repeat until empty cell is found.
- Write it:

```

void insertQuadratic(vector<int>& table, int key, int emptyValue) {
    int size = table.size();
    int index = key % size;
    int i = 0;

    while (table[(index + i * i) % size] != emptyValue) {
        i++;
    }

    table[(index + i * i) % size] = key;
}

```

Quadratic Probing Memory Aid: "Square the Step"

- Collision step is not +1 every time.
- It jumps by square offsets: 1, 4, 9, ...

9.11 Chaining

- From slides:
 - Each cell points to a linked list of records with the same hash value.
 - When collision occurs, create a new node.
 - Store and link the new value.
 - Worst-case search time is $O(n)$.

- Advantages:
 - Simple to implement.
 - Hash table never fills up completely.
 - Less sensitive to hash function/load factor.
 - Useful when number/frequency of keys is unknown.
 - Deletion is easier.
- Disadvantages:
 - Cache performance is not good.
 - Some table slots may never be used.
 - Long chains can make search $O(n)$.
 - Extra space needed for links.

- Write it:

```
vector<vector<int>> hashTable(10);

void insertChaining(int key) {
    int index = key % hashTable.size();
    hashTable[index].push_back(key);
}

bool searchChaining(int key) {
    int index = key % hashTable.size();

    for (int value : hashTable[index]) {
        if (value == key) return true;
    }

    return false;
}
```

Chaining Memory Aid: "Same Index, Same Chain"

- If keys collide, link them in the same bucket chain.

9.12 Double Hashing

- From slides:
 - Open addressed collision resolving technique.
 - Applies a second hash function when collision occurs.
- Formula from slides:
 - $\text{hash1}(\text{key}) = \text{key} \% \text{tableSize}$
 - $\text{hash2}(\text{key}) = \text{PRIME} - (\text{key} \% \text{PRIME})$
 - New index:
 - $(\text{hash1}(\text{key}) + i * \text{hash2}(\text{key})) \% \text{tableSize}$
 - Increase i when collision occurs.
- Step-by-step:
 1. Compute $\text{hash1}(\text{key})$.
 2. If slot is empty, insert.
 3. If collision occurs, compute $\text{hash2}(\text{key})$.
 4. Try $(\text{hash1} + i * \text{hash2}) \% \text{tableSize}$.
 5. Increase i until an empty slot is found.

- Write it:

```
int hash2(int key, int prime) {
    return prime - (key % prime);
}
```

```

}

void insertDoubleHash(vector<int>& table, int key, int emptyValue, int prime) {
    int size = table.size();
    int h1 = key % size;
    int h2 = prime - (key % prime);
    int i = 0;

    int index = h1;
    while (table[index] != emptyValue) {
        i++;
        index = (h1 + i * h2) % size;
    }

    table[index] = key;
}

```

Double Hashing Memory Aid: "Two Hashes, One Home"

- First hash finds the home.
- Second hash decides jump size.
- Repeated jumps find an empty home.

Exam-Focused Master Checklist

- ADT:
 - Can explain "what, not how".
 - Can list values, operations, and behavior.
- Big O:
 - Can drop constants and low-order terms.
 - Can identify $O(1)$, $O(n)$, $O(n^2)$, $O(\log n)$, $O(n \log n)$.
- Vector/STL:
 - Can use `vec.size()` and iterators.
 - Can explain why traversal is $O(n)$.
- Singly linked list:
 - Can create node.
 - Can insert at empty, beginning, middle, end.
 - Can delete at beginning, middle, end.
 - Can traverse and reverse.
- Doubly linked list:
 - Can maintain both `prev` and `next`.
 - Can insert/delete without breaking both directions.
 - Can traverse forward/backward and reverse.
- Stack:
 - Can explain LIFO.
 - Can use STL stack.
 - Can implement linked-list stack.
 - Can check balanced brackets.
 - Can convert infix to postfix.
 - Can evaluate postfix.
- Queue:

- Can explain FIFO.
- Can use STL queue.
- Can implement linked-list queue.
- Tree:
 - Can define root, edge, child, sibling, leaf, internal, depth, height, level.
 - Can insert/delete/search BST.
 - Can write preorder, inorder, postorder.
 - Can connect expression tree postorder with postfix.
- Graph:
 - Can define vertices, edges, order, size.
 - Can represent graph using adjacency list and matrix.
 - Can apply Dijkstra.
 - Can apply Kruskal.
- Sorting:
 - Can trace insertion, selection, bubble, heap, quick.
 - Can explain merge sort as $O(n \log n)$ because it is named in Chapter 2.
- Searching:
 - Can compare sequential and binary search.
 - Can use hashing and collision resolution.